SWIFTDROP

Software Requirements Specification

Parcel Delivery & Courier Management System

Document ID: SWD-SRS-001
Version: 1.0 — Proposed Baseline
Status: Draft for team review
Prepared: 21 October 2026

Developer-facing requirements baseline aligned to the supplied assignment brief and agreed architecture.

Contents

Document Control

1. Introduction

2. Product Description

3. System Context and Components

4. Functional Requirements

5. Business Rules and State Machine

6. External Interface Requirements

7. Data Requirements

8. Security and Privacy Requirements

9. Non-Functional Requirements

10. Event Processing and Failure Recovery

11. Deployment and Operations

12. Testing and Verification

13. Design Decisions and Developer Contracts

14. Risks and Mitigations

15. Project Priorities and Acceptance

16. Requirements Traceability Matrix

17. Deliverables and Open Items

Document Control

Field	Value
Document title	SwiftDrop Parcel Delivery & Courier Management System — Software Requirements Specification
Document ID	SWD-SRS-001
Version	1.0 — Proposed Baseline
Status	Draft for team review and approval
Prepared for	SwiftDrop academic group project
Source baseline	Group Assignment.pdf supplied in the conversation
Architecture baseline	Go modular-monolith Core API; separate Tracking API, Outbox Relay, Event Worker, and Python AI Assistant; PostgreSQL; RabbitMQ; Kubernetes/Helm bonus deployment
Audience	Frontend, backend, QA, DevOps, security, and documentation team members
Approval	To be completed by project team / lecturer

Document conventions: “Shall” means a mandatory requirement in this baseline. “Should” means a recommended implementation. “May” means optional. Requirements labelled BASELINE are proposed engineering decisions where the assignment did not prescribe an exact value; the team should approve or revise them before implementation.

1. Introduction

1.1 Purpose

This SRS defines the functional, interface, data, security, operational, and quality requirements for SwiftDrop, a parcel delivery and courier management platform. It is intended to give developers and testers a shared implementation contract, reduce ambiguous decisions, and provide verifiable acceptance criteria.

1.2 Scope

SwiftDrop replaces phone-call and spreadsheet-based parcel booking and tracking with a web application and secured merchant API. Customers create and track parcels; merchants submit orders programmatically and receive status webhooks; dispatchers assign deliveries; couriers update delivery progress and proof; administrators manage users, merchants, delivery zones, pricing, and reports. The system sends status-triggered notifications and supports a real-time tracking experience. An AI assistant is an optional assignment bonus and is included in this architecture baseline.

1.3 Goals

- Provide secure customer, merchant, courier, dispatcher, and administrator workflows.
- Maintain one authoritative parcel state and auditable status history.
- Provide timely status updates to web clients and external merchants.
- Prevent duplicate or lost business events through a transactional outbox and idempotent consumers.
- Support local development with Docker Compose and deployment to Kubernetes for the bonus requirement.
- Provide tests, deployment documentation, architecture/design artifacts, and contribution evidence.

1.4 In-scope capabilities

- Email/password account signup and login; Google sign-in using OAuth 2.0 / OpenID Connect.
- Customer parcel booking, fee estimate, tracking, history, and receipt download.
- Merchant application/approval, secured API order creation, dashboard, and webhook subscriptions.
- Courier assigned-delivery list, status updates, delivery notes/proof, and failed-delivery reasons.
- Dispatcher assignment and workload monitoring; administrator management of users, merchants, zones, and pricing.
- Versioned REST APIs, JWT-based application authentication, role/resource-based authorization.
- RabbitMQ-based asynchronous status events, notification delivery, merchant webhooks, retries, and dead-letter handling.
- Operational reconciliation, metrics/logs/health probes, and Kubernetes deployment.
- AI assistant calling authorized backend APIs through controlled MCP tools.

1.5 Out of scope unless separately approved

- Live GPS tracking of courier location, route optimization, maps/navigation, and driver telematics.
- Online payment processing, refunds, invoices, accounting, and tax integrations.
- Automatic address geocoding/validation, unless a provider is explicitly selected.
- Customer mobile native applications; the required client is a responsive web application.
- Guaranteed email/SMS delivery beyond the chosen provider’s guarantees.

• Multi-region active-active deployment, disaster recovery with a defined RPO/RTO, and production-grade 24/7 operations beyond the project needs.

1.6 Definitions and abbreviations

Term	Meaning
API	Application Programming Interface
RBAC	Role-Based Access Control
JWT	JSON Web Token
OIDC	OpenID Connect; identity layer over OAuth 2.0
SSE	Server-Sent Events; server-to-browser one-way streaming
Outbox	Database records written in the same transaction as a business change and published asynchronously
Relay	Process that publishes pending outbox events to RabbitMQ
Event Worker	Consumer that executes notifications and webhook delivery
DLQ	Dead-letter queue for messages that cannot be processed normally
Idempotency	Repeated processing of the same request/event does not create repeated business effects
MCP	Model Context Protocol; structured interface through which an AI model can call approved tools
SLO	Service-level objective; a target for a quality attribute
P0/P1/P2	Priority levels used for defect triage; defined in Section 15

2. Product Description

2.1 Product perspective

SwiftDrop is a web-based enterprise application composed of a React frontend, a Go Core API implemented as a modular monolith, a separate Go Tracking API, a Go Outbox Relay, a Go Event Worker, a Python AI Assistant, PostgreSQL, RabbitMQ, and gateway/Kubernetes infrastructure. The AI Assistant is a separate workload; its MCP tools may initially be packaged with that service. The Core API owns business mutations. PostgreSQL is the authoritative store. RabbitMQ transports asynchronous work and is not the source of truth for parcel state.

2.2 User classes

User class	Main capabilities	Key restrictions
Customer	Manage own profile; estimate/create parcels; view own parcel history; track by public tracking number; download eligible receipts	May not view another customer's private account/history or change operational parcel state
Merchant user / API client	Create orders for approved merchant; view merchant orders; configure approved webhook endpoints; receive webhook events	All data scoped to the authenticated merchant tenant; cannot access other merchants' orders
Courier	View assigned active deliveries; record permitted status changes; add notes/proof; record failure reason	May act only on assigned deliveries; cannot assign parcels or alter pricing
Dispatcher	View unassigned parcels; assign/reassign couriers; monitor workload; review delivery operations	Cannot manage privileged identity/security settings unless also granted admin permission
Administrator	Manage users, merchant approvals, zones/pricing, system settings, and reports; perform audited corrections	Sensitive actions require explicit permission and audit record
Public tracking visitor	View limited parcel status using a valid public tracking number	No private addresses, phone numbers, recipient data, or proof files unless separately authorized
AI Assistant user	Ask questions answered using approved backend tools	Assistant inherits the authenticated user's permissions; model output is never an authorization decision

2.3 Operating environment

- Modern desktop and mobile browsers; responsive layout.
- Frontend: React + TypeScript + Vite; Tailwind CSS; browser APIs including EventSource for SSE.
- Backend: Go HTTP services; Python FastAPI AI service.
- Data: PostgreSQL; messaging: RabbitMQ.
- Container runtime: Docker; local orchestration: Docker Compose.
- Deployment: Kubernetes with Services, ConfigMaps, Secrets, Ingress/Gateway routing, Deployments, CronJob, HPA, KEDA, and Helm charts where implemented.

2.4 Architectural principles

- Modular monolith for Core API: one deployable unit, separated internal modules, no network calls between modules.
- Separate deployments only where isolation/scaling/runtime needs justify them: Tracking API, Outbox Relay, Event Worker, AI Assistant.
- REST/JSON is the default synchronous interface; RabbitMQ is used for asynchronous event-driven work; SSE is used for browser live tracking.
- Business state and outbox event are committed atomically in PostgreSQL.
- At-least-once message delivery is assumed; consumers must be idempotent.
- Backend services enforce authorization even if the gateway or frontend has already checked access.
- External providers are accessed through adapters/interfaces to avoid vendor coupling.

3. System Context and Components

3.1 Component responsibilities

Component	Responsibilities	Does not own
Frontend	Role-specific user experience; form validation; API calls; SSE client; display errors	Security decisions; authoritative business rules
Ingress/API Gateway	TLS termination, route mapping, request limits, gateway policies, rate limiting	Domain authorization or parcel state
Core API	Identity, access, merchant approval, parcel lifecycle, pricing, dispatch, proof records, admin, outbox writes	Asynchronous notification delivery execution
Tracking API	Public/private tracking queries, status history reads, SSE subscriptions	Parcel mutation; independent authoritative state
Outbox Relay	Claim pending outbox rows, publish persistent events to RabbitMQ, record publication result	Customer notification or webhook execution
Event Worker	Consume event messages; create/send notification and webhook deliveries; retry and DLQ handling	Changing authoritative parcel status except via authorized Core API operation
AI Assistant + MCP tools	Interpret user question; call allow-listed backend tools; summarize tool results	Direct database access or authorization bypass
PostgreSQL	Authoritative relational data, transactions, outbox, audit/delivery records	Message transport
RabbitMQ	Route/buffer asynchronous event messages	Authoritative parcel state
Reconciliation CronJob	Detect stale outbox/delivery records and inconsistencies; safely schedule recovery or alert	Routine direct publication that bypasses the relay
Prometheus/Grafana/logging	Metrics, dashboards, diagnostics	Business transactions

3.2 Deployment units

- core-api: one image/deployment for the modular monolith.
- tracking-api: independent deployment and read-only PostgreSQL role.
- outbox-relay: independent deployment; may scale horizontally with safe row claiming.
- event-worker: independent deployment; may scale based on RabbitMQ backlog via KEDA.
- ai-assistant: independent Python deployment; MCP tools included initially.
- frontend: static assets served by a web server/container or static host; for the Kubernetes bonus, deploy it in the cluster.

- postgresql and rabbitmq: local containers for Compose; in Kubernetes, deploy in-cluster if required by the rubric, or use managed equivalents if allowed.
- reconciliation: Kubernetes CronJob using a Go application image and a dedicated command.

3.3 Trust boundaries

- Public internet to gateway: untrusted input; TLS, rate limiting, size limits, authentication where required.
- Gateway to application services: private cluster networking; backend authorization still mandatory.
- Application services to PostgreSQL/RabbitMQ: service credentials, least privilege, TLS when available.
- Event Worker to external providers: outbound-only calls with timeouts and destination validation.
- AI Assistant to backend tools: authenticated, allow-listed calls; no direct database credentials.
- Public tracking: tracking number is an identifier, not proof of identity; only a privacy-safe subset of fields is returned.

4. Functional Requirements

Requirement identifiers are stable references for implementation and test cases. Unless stated otherwise, all API operations must validate input, apply authorization, return consistent errors, and produce appropriate audit/operational logs without exposing secrets.

4.1 Identity and access management

ID	Requirement	Priority
FR-IAM-001	The system shall allow a user to register with required profile details and a unique verified email address, subject to the configured verification policy.	P1
FR-IAM-002	The system shall authenticate local accounts using securely hashed passwords and shall never store plaintext passwords.	P1
FR-IAM-003	The system shall support Google sign-in through OAuth 2.0/OIDC and validate issuer, audience, signature, expiry, state, and nonce as applicable.	P1
FR-IAM-004	The system shall issue application access tokens after successful authentication and validate token signature, expiry, issuer/audience where configured, and required claims.	P1
FR-IAM-005	The system shall support role-based authorization for customer, merchant, courier, dispatcher, and administrator capabilities.	P1
FR-IAM-006	The system shall enforce object-level authorization so users can access only resources they own or are assigned to, unless an explicit permission allows broader access.	P1
FR-IAM-007	The system shall allow administrators with the relevant permission to disable accounts and revoke or invalidate sessions/refresh tokens according to the selected token strategy.	P1
FR-IAM-008	The system shall return generic authentication failures and shall not reveal whether an arbitrary email address has an account.	P1
FR-IAM-009	The system shall audit security-sensitive actions including role changes, account disablement, merchant approval, and administrative status correction.	P1

4.2 Customer and parcel booking

ID	Requirement	Priority
FR-PAR-001	The customer shall be able to create a parcel booking with pickup/drop-off contact and address details, parcel size/dimensions where required, weight, and service option.	P1
FR-PAR-002	The system shall provide an estimated delivery fee before the customer confirms the booking.	P1
FR-PAR-003	The system shall identify the pricing rule/version used and store the confirmed price snapshot with the booking.	P1
FR-PAR-004	On successful booking, the system shall generate a unique, non-guessable tracking number and return a booking confirmation.	P1
FR-PAR-005	The system shall reject invalid or unsupported parcel dimensions, weight, addresses, service zones, and missing required fields with field-level validation errors.	P1
FR-PAR-006	The system shall support idempotent booking creation so a client retry with the same idempotency key does not create a second parcel.	P1
FR-PAR-007	The customer shall be able to list and view their own parcel bookings and status history.	P1
FR-PAR-008	The customer shall be able to download a receipt for an eligible booking; the receipt shall include parcel reference, booking time, price, service details, and current/terminal status as appropriate.	P1
FR-PAR-009	The system shall not expose private address/contact details through public tracking responses.	P1

4.3 Parcel lifecycle and status history

BASELINE lifecycle: CREATED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED. FAILED is a terminal outcome for a delivery attempt or parcel only according to the approved operational policy. A failed attempt may be retried by an authorized dispatcher/courier workflow; if retries are allowed, model attempt outcome separately from final parcel lifecycle rather than silently reopening a terminal parcel.

ID	Requirement	Priority
FR-LIFE-001	The system shall maintain the current parcel status and append-only status history containing status, timestamp, actor, optional note, and relevant attempt/assignment reference.	P1
FR-LIFE-002	The system shall validate every status transition against the configured state machine and reject illegal transitions.	P1
FR-LIFE-003	The system shall record actor identity and server-side timestamp for every operational status update.	P1

ID	Requirement	Priority
FR-LIFE-004	The system shall atomically persist the parcel status change, history entry, and corresponding outbox event in one PostgreSQL transaction.	P1
FR-LIFE-005	The system shall prevent duplicate status changes from producing duplicate history or downstream side effects when the same idempotency key/event is retried.	P1
FR-LIFE-006	The system shall require a failure reason when a courier records a failed delivery attempt.	P1
FR-LIFE-007	The system shall permit authorized correction of erroneous status data only through an audited operation; ordinary users shall not edit history directly.	P1

4.4 Tracking and real-time updates

ID	Requirement	Priority
FR-TRK-001	The system shall provide a mobile-friendly public tracking page addressable by a tracking number.	P1
FR-TRK-002	The public tracking endpoint shall return only the approved privacy-safe fields: tracking number, current status, relevant status timestamps, and an appropriate delivery summary.	P1
FR-TRK-003	Authenticated customers shall be able to view the full permitted status history for their own parcels.	P1
FR-TRK-004	The Tracking API shall provide SSE updates when the parcel status changes, subject to connection and authorization policy.	P1
FR-TRK-005	After an SSE reconnect or dropped event, the client shall fetch the current authoritative state; event delivery alone shall not be treated as a durable history source.	P1
FR-TRK-006	The system shall support multiple Tracking API replicas and a shared event fan-out mechanism if required to deliver updates to clients connected to different replicas.	P1
FR-TRK-007	The tracking page shall display a clear unavailable/unknown result for invalid tracking numbers without exposing database or internal error details.	P1

4.5 Merchant management and integration

ID	Requirement	Priority
FR-MER-001	A business shall be able to submit a merchant registration/application for administrative approval.	P1
FR-MER-002	Only an approved and enabled merchant shall be able to create production parcel orders through the merchant API.	P1
FR-MER-003	The system shall authenticate merchant API clients using revocable API keys or OAuth 2.0 client credentials; the selected method shall be documented and implemented consistently.	P1
FR-MER-004	The system shall scope every merchant order query and mutation to the authenticated merchant tenant.	P1
FR-MER-005	The merchant API shall support versioned order creation and return a stable parcel identifier and tracking number on success.	P1
FR-MER-006	The system shall support idempotency for merchant order creation to prevent duplicate orders on retries.	P1
FR-MER-007	The merchant dashboard shall list orders and their current delivery states, with appropriate filtering and pagination.	P1
FR-MER-008	An authorized merchant user shall be able to register and manage webhook endpoints subject to URL validation and secret management policy.	P1
FR-MER-009	The system shall send configured webhook events after relevant status changes through the asynchronous worker pipeline.	P1
FR-MER-010	Webhook requests shall include event ID, event type, schema version, occurrence time, and parcel reference; requests shall be signed using a configured secret.	P1
FR-MER-011	The system shall record each webhook delivery attempt, HTTP outcome, attempt time, next retry time, and final state without storing sensitive response content unnecessarily.	P1

4.6 Courier operations

ID	Requirement	Priority
FR-COU-001	An authenticated courier shall be able to view their assigned deliveries for the current day and relevant active assignments.	P1
FR-COU-002	A courier shall be able to view operational details needed to complete an assigned delivery, limited to their assigned work.	P1
FR-COU-003	A courier shall be able to update status only for parcels assigned to them, unless an explicit elevated permission applies.	P1
FR-COU-004	The system shall capture a delivery note and/or proof metadata when required by the configured status policy.	P1
FR-COU-005	The system shall require a failure reason for failed delivery attempts and record the attempt time and actor.	P1
FR-COU-006	The system shall reject stale or conflicting updates when the parcel state/assignment changed since the courier loaded it, returning a conflict response and current state where safe.	P1
FR-COU-007	If photo/signature proof is implemented, the system shall store files in private object storage or a defined local-development substitute and store only metadata/reference in PostgreSQL.	P2

4.7 Dispatcher and administration

ID	Requirement	Priority
FR-ADM-001	A dispatcher shall be able to list unassigned parcels eligible for dispatch.	P1
FR-ADM-002	A dispatcher shall be able to assign an eligible parcel to an enabled courier.	P1
FR-ADM-003	The system shall prevent concurrent assignments from leaving a parcel with conflicting active courier assignments.	P1
FR-ADM-004	The dispatcher interface shall show each courier's active workload count and assigned parcels.	P1
FR-ADM-005	An authorized administrator shall be able to review, approve, reject, suspend, or disable merchant accounts with an audit trail.	P1
FR-ADM-006	An authorized administrator shall be able to manage users and role assignments within permission boundaries.	P1
FR-ADM-007	An authorized administrator shall be able to manage delivery zones and versioned pricing rules with effective dates.	P1
FR-ADM-008	The system shall provide basic reports for deliveries per day and delivery success/failure rate over a selected period.	P1
FR-ADM-009	Administrative parcel corrections shall require a reason and create an audit record.	P1

4.8 Notifications and asynchronous processing

ID	Requirement	Priority
FR-EVT-001	A successful business mutation that requires asynchronous follow-up shall insert an outbox event in the same transaction as the business mutation.	P1
FR-EVT-002	The Outbox Relay shall publish pending events to RabbitMQ using persistent messages and publisher confirms.	P1
FR-EVT-003	The relay shall mark an event published only after receiving a broker confirmation; failures shall leave it eligible for retry or explicit investigation.	P1
FR-EVT-004	Every event shall have a stable unique event ID, event type, schema version, occurrence timestamp, and aggregate/parcel reference.	P1
FR-EVT-005	The Event Worker shall consume status events and create the required notification and merchant webhook delivery tasks.	P1
FR-EVT-006	Consumers shall be idempotent and shall tolerate duplicate delivery.	P1
FR-EVT-007	Transient failures shall be retried with bounded exponential backoff and jitter; repeated failures shall be routed to a dead-letter or equivalent investigation path.	P1
FR-EVT-008	The system shall provide at least one notification channel: email provider integration or a mock SMS/email adapter suitable for demonstration.	P1
FR-EVT-009	The system shall expose operational counts for pending outbox events, publish failures, queue backlog, consumer failures, webhook attempts, and DLQ messages.	P1
FR-EVT-010	The system shall not claim exactly-once delivery across PostgreSQL, RabbitMQ, and external providers; it shall implement at-least-once delivery with idempotent effects.	P1

4.9 Reconciliation

ID	Requirement	Priority
FR-REC-001	A scheduled reconciliation command shall identify outbox records that exceed the configured publication threshold.	P1
FR-REC-002	The reconciliation job shall identify webhook/notification deliveries whose retry deadlines have elapsed and safely schedule eligible retries.	P1
FR-REC-003	The reconciliation job shall detect configured operational inconsistencies, such as a parcel missing expected status-history or assignment records.	P1
FR-REC-004	Reconciliation actions shall be idempotent, bounded by batch size, logged, measurable, and auditable where they change state.	P1
FR-REC-005	The reconciliation job shall not bypass the normal outbox publication flow for routine event publication.	P1

4.10 AI Delivery Assistant (bonus)

ID	Requirement	Priority
FR-AI-001	The AI Assistant shall answer parcel tracking questions by calling an approved backend tracking tool rather than inventing status.	P2
FR-AI-002	The AI Assistant shall estimate delivery fees by calling the Core API pricing endpoint with validated parcel inputs.	P2
FR-AI-003	AI tools shall enforce the caller's authentication and authorization context; the LLM shall not decide access rights.	P2
FR-AI-004	The AI Assistant shall use an allow-list of tool names and schemas and shall validate tool arguments before backend calls.	P2
FR-AI-005	The AI Assistant shall not have direct database access or be given secrets in model prompts.	P2
FR-AI-006	The AI Assistant shall state when data is unavailable and shall not fabricate tracking status, fee quotes, or delivery dates.	P2
FR-AI-007	Initial MCP tools shall be read-only: track_parcel, get_delivery_history, estimate_delivery_fee.	P2
FR-AI-008	The system shall apply rate limits, timeouts, token/cost limits, and safe error handling to AI requests.	P2

5. Business Rules and State Machine

5.1 Parcel status rules

Status	Meaning	Allowed actor / rule
CREATED	Booking accepted; awaiting pickup/dispatch	System on booking creation
PICKED_UP	Courier has collected the parcel	Assigned courier or authorized operations user
IN_TRANSIT	Parcel is moving through delivery operations	Assigned courier/authorized operations; transition must be valid
OUT_FOR_DELIVERY	Parcel is with courier for final delivery attempt	Assigned courier/authorized operations
DELIVERED	Successful terminal delivery outcome	Assigned courier with required delivery evidence, or authorized correction
FAILED	Delivery attempt failed; terminality policy must be explicitly selected	Assigned courier must provide failure reason; retries require explicit attempt model/policy

BASELINE decision required before coding: model FAILED as a failed attempt with a possible later reattempt, or as a terminal parcel status. Recommended approach is to represent delivery attempts separately and allow the parcel to return to an eligible operational state only through an authorized dispatcher workflow. Do not silently allow arbitrary transitions.

5.2 Pricing rules

- All money values shall be stored using fixed precision (e.g., integer minor units or PostgreSQL NUMERIC), never binary floating-point.
- A quote shall include currency, amount, pricing rule version, and expiry or validity policy if quotes can expire.
- Final booking shall persist the price accepted by the customer/merchant.
- Zone eligibility and weight/dimension limits shall be validated before quote acceptance.
- Pricing calculation shall be deterministic for the same inputs and rule version.

5.3 Assignment rules

- A parcel may have at most one active courier assignment at a time unless the domain model explicitly supports multi-courier handling.
- Disabled couriers cannot receive new assignments.
- Reassignment must preserve prior assignment history and record the actor/reason.
- Active workload is derived from assignments and current parcel state using a documented definition.

5.4 Idempotency and event ordering

- Clients may send an Idempotency-Key for create operations; the server stores the key scoped to caller and operation with request hash and resulting resource reference.
- Reusing a key with the same request returns the prior result; reusing it with a different request returns a conflict.
- Events include aggregate ID and monotonically increasing aggregate version or sequence where needed.
- Consumers must tolerate duplicate events and ignore/reconcile stale versions where the use case requires ordering.
- RabbitMQ ordering is not assumed across multiple queues or competing consumers.

6. External Interface Requirements

6.1 REST API conventions

- Base path: /api/v1 for versioned APIs. Tracking may be exposed under /api/v1/tracking or a documented public route.
- Content type: application/json; UTF-8.
- Use ISO 8601/RFC 3339 timestamps in UTC in API payloads.
- Use stable resource IDs (UUID recommended) and separate public tracking numbers.
- List endpoints support bounded pagination and documented sorting/filtering.

- Use standard HTTP semantics: 201 created, 200 success, 202 accepted for asynchronous work where applicable, 400 validation, 401 unauthenticated, 403 unauthorized, 404 absent/not visible, 409 conflict, 422 semantic validation if selected, 429 rate limited, 500 unexpected failure, 503 dependency unavailable.
- Centralized error format: code, message, request_id, and optional field_errors; never include stack traces, SQL, credentials, tokens, or internal hostnames.
- Document API contracts with OpenAPI and keep them under source control.

6.2 Proposed endpoint inventory

Method / path	Purpose	Access
POST /api/v1/auth/register	Register local account	Public, rate limited
POST /api/v1/auth/login	Authenticate local account	Public, rate limited
GET /api/v1/auth/google	Begin Google OIDC flow	Public
GET /api/v1/auth/google/callback	Handle identity provider callback	Public flow with state/nonce validation
POST /api/v1/auth/refresh	Refresh session/token if selected	Refresh credential required
POST /api/v1/auth/logout	Revoke refresh/session if implemented	Authenticated
POST /api/v1/pricing/quotes	Calculate delivery fee quote	Customer or approved merchant
POST /api/v1/parcels	Create customer parcel booking	Customer
GET /api/v1/parcels	List caller-visible parcels	Customer/merchant; tenant scoped
GET /api/v1/parcels/{id}	Get authorized parcel details	Owner/merchant/assigned courier/authorized staff
GET /api/v1/parcels/{id}/history	Get authorized status history	Authorized caller
GET /api/v1/parcels/{id}/receipt	Download/generate receipt	Owner/authorized merchant
GET /api/v1/tracking/{trackingNumber}	Privacy-safe public tracking lookup	Public, rate limited
GET /api/v1/tracking/{trackingNumber}/events	SSE live status events	Public-safe or authenticated scope
POST /api/v1/merchant/orders	Create parcel through merchant API	Approved merchant client
GET /api/v1/merchant/orders	List merchant orders	Merchant tenant
POST /api/v1/merchant/webhooks	Register webhook endpoint	Merchant admin
GET /api/v1/courier/deliveries	List courier's assignments	Courier
POST /api/v1/courier/parcels/{id}/status	Update parcel status/attempt	Assigned courier
GET /api/v1/dispatch/unassigned	List eligible unassigned parcels	Dispatcher
POST /api/v1/dispatch/assignments	Assign parcel to courier	Dispatcher
GET /api/v1/dispatch/workloads	Courier workload summary	Dispatcher
GET /api/v1/admin/reports/deliveries	Delivery counts and success/failure report	Admin/authorized dispatcher
POST /api/v1/assistant/chat	Ask AI assistant	Authenticated user, rate limited
GET /health/live	Process liveness	Internal/platform
GET /health/ready	Readiness check	Internal/platform
GET /metrics	Prometheus metrics	Internal monitoring network

6.3 Representative request/response contracts

Quote request (illustrative): {"pickup_zone_id":"...","dropoff_zone_id":"...","weight_kg":2.0,"length_cm":30,"width_cm":20,"height_cm":10,"service_level":"STANDARD"}. Quote response: {"quote_id":"...","currency":"LKR","amount_minor":125000,"pricing_version":"...","valid_until":"..."}. Exact fee formula and quote validity must be configured and documented.

Parcel creation request: {"quote_id":"...","pickup":{"name":"...","phone":"...","address_lines":["..."],"city":"...","zone_id":"..."},"dropoff":{"name":"...","phone":"...","address_lines":["..."],"city":"...","zone_id":"..."},"parcel":{"weight_kg":2.0,"dimensions_cm":{"length":30,"width":20,"height":10},"description":"..."}}. Response returns parcel ID, tracking number, status CREATED, and accepted price.

Status update request: {"status":"IN_TRANSIT","note":"...","delivery_attempt_id":"...","proof_reference":"..."}. Server derives actor identity from credentials; client must not set actor_user_id. Failure requests must include a configured failure_reason_code.

6.4 Webhook contract

Webhook body baseline: event_id, event_type, schema_version, occurred_at, parcel_id, tracking_number, parcel_version, status, and minimal merchant-safe payload. Include a signature header derived from the raw request body and secret, plus timestamp/replay protection. Never send unrelated customer data. Merchant endpoints must return a 2xx response to indicate acceptance; all other responses/timeouts follow the retry policy.

6.5 SSE contract

SSE events shall include an event ID and event type, with a compact payload containing the parcel ID/tracking number, current status, occurrence timestamp, and version. The client shall treat SSE as a notification that state changed, not as the only durable source. On reconnect, it shall re-fetch current state. If Last-Event-ID replay is supported, its retention/window must be documented.

6.6 External integrations

Integration	Protocol / purpose	Failure behavior
Google identity	OAuth 2.0/OIDC authorization-code flow	Show safe login failure; validate state/nonce; never accept unvalidated identity claims
Email/SMS provider	HTTPS API or mock adapter	Timeout, bounded retries, provider ID tracking; DLQ/investigation after exhaustion
Merchant webhook	Outbound HTTPS POST	Signature, timeout, retries, per-endpoint delivery history, SSRF defenses
LLM provider	HTTPS API from AI Assistant	Timeout, token/cost limits, safe fallback; no sensitive data beyond necessary tool context
RabbitMQ	AMQP 0-9-1	Publisher confirms, durable queues, consumer acknowledgments, retry/DLQ

7. Data Requirements

7.1 Logical entities

Entity	Key fields / purpose
users	id, email, password_hash nullable for federated-only accounts, display_name, phone, status, created_at
roles / user_roles	role definitions and user-role assignment
oauth_identities	user_id, provider, provider_subject; unique provider + subject
merchants	id, legal/display name, status pending/approved/rejected/suspended, contact, approval metadata
merchant_api_credentials	merchant_id, key_hash or client identifier, scopes, expiry/revocation metadata
couriers	user_id, operational status, optional employee code
addresses / parcel_addresses	parcel_id, type pickup/dropoff, contact name/phone, address lines, city/zone; access restricted
parcels	id, tracking_number unique, customer_id or merchant_id, current_status, parcel dimensions/weight, service level, accepted price snapshot, version, timestamps
parcel_status_history	id, parcel_id, from_status, to_status, actor_id, note, reason code, occurred_at, parcel_version
delivery_attempts	id, parcel_id, courier_id, attempt_number, outcome, failure_reason, proof metadata, attempted_at
courier_assignments	id, parcel_id, courier_id, assigned_by, assigned_at, ended_at, active flag/status, reason
delivery_zones	id, name, code, enabled, coverage metadata
pricing_rules	id, version, zone pair/criteria, base fee, weight/dimension bands, effective_from/to, enabled
pricing_quotes	id, requester/tenant, input snapshot, rule version, amount/currency, expiry, accepted parcel reference
outbox_events	id/event_id, aggregate_type/id/version, event_type, schema_version, payload, created_at, published_at, attempt_count, last_error, status
notification_deliveries	id, event_id, recipient, channel, status, provider_message_id, attempts, next_attempt_at, timestamps
webhook_endpoints	id, merchant_id, url, secret reference, subscribed event types, enabled
webhook_deliveries	id, endpoint_id, event_id, status, attempts, last status code, next_attempt_at, timestamps
idempotency_keys	caller scope, operation, key, request hash, response/resource reference, expiry
audit_logs	actor, action, target type/id, reason, timestamp, request ID, minimal safe metadata
ai_conversations (optional)	minimal conversation metadata; avoid storing prompts or personal data unless necessary and disclosed

7.2 Data integrity requirements

- Use UUIDs or another stable internal identifier; public tracking numbers must be unique and difficult to enumerate.
- Use foreign keys and unique constraints for key relationships.
- Use transactions for parcel status + status history + outbox insertion.
- Use optimistic versioning or row locks to prevent conflicting concurrent status/assignment updates.
- Use UTC timestamps in storage and convert to local display time in the UI.
- Store money in integer minor units or exact NUMERIC with a documented currency scale.
- Index tracking_number, customer_id, merchant_id, current_status, courier assignment, event publication status, and due retry timestamps according to measured queries.
- Separate immutable history/audit records from mutable current-state fields.
- Use soft-disable/status fields for accounts and merchants where auditability is required; define deletion/retention policy for personal data.

7.3 Data retention and backup

BASELINE proposal: define retention periods for audit records, delivery proof, webhook bodies/responses, and personal data before production. The academic implementation shall document a backup/restore procedure for PostgreSQL and demonstrate that migrations can rebuild a clean schema. Secrets and proof files shall not be included in database dumps or logs unless explicitly intended and protected.

8. Security and Privacy Requirements

ID	Requirement	Priority
SEC-001	All external traffic shall use HTTPS in deployed environments; local development may use HTTP only on loopback/private networks.	P1
SEC-002	Passwords shall be hashed with Argon2id or another approved password hashing algorithm using current secure parameters.	P1
SEC-003	Access tokens shall have bounded lifetimes; signing keys/secrets shall be supplied via secret configuration and rotated under a documented procedure.	P1
SEC-004	The system shall validate authentication and authorization on every protected operation, including object-level and tenant-level access.	P1
SEC-005	API keys shall be stored as hashes where feasible, shown only at creation, revocable, scoped, and auditable.	P1
SEC-006	The system shall validate all input at the API boundary and use parameterized database queries.	P1
SEC-007	The system shall apply rate limits to login, public tracking, merchant API, webhook registration, and AI endpoints.	P1
SEC-008	Sensitive values including passwords, access tokens, API keys, webhook secrets, full payment credentials, and unnecessary personal data shall never be written to logs.	P1
SEC-009	Public tracking responses shall not disclose full pickup/drop-off addresses, phone numbers, recipient names, private notes, or proof files.	P1
SEC-010	Webhook destinations shall be validated to mitigate SSRF, including restrictions on loopback, private, link-local, and cloud metadata addresses; DNS rebinding risks shall be considered.	P1
SEC-011	Webhook signatures shall be verified by recipients using a documented algorithm; the sender shall support timestamp/replay protection.	P1
SEC-012	File uploads, if implemented, shall enforce type/size limits, random object names, malware scanning where available, and private access controls.	P2
SEC-013	CORS shall allow only configured frontend origins; CSRF protections shall be used if cookie-based authentication is chosen.	P1
SEC-014	The AI Assistant shall use an allow-list of tools, validate all arguments, and preserve user authorization context; model output shall not execute arbitrary code or SQL.	P2
SEC-015	Administrative actions and security-sensitive changes shall be audit logged with actor, action, target, timestamp, request ID, and reason where relevant.	P1
SEC-016	Container images shall run as non-root where practical, use pinned versions, and avoid embedding credentials or build-time secrets.	P1
SEC-017	Kubernetes RBAC and network exposure shall follow least privilege; only intended public entry points shall be externally reachable.	P1

8.1 Authentication baseline decision

Recommended initial design: short-lived signed JWT access token plus a refresh mechanism stored in a secure HttpOnly cookie for browser sessions, if the team chooses cookie-based sessions. Merchant API credentials use a separate machine-to-machine authentication path. The exact access-token lifetime, refresh rotation, logout semantics, and storage strategy are configuration/design decisions that must be recorded before implementation. Do not mix browser-session credentials with merchant API keys.

9. Non-Functional Requirements

9.1 Performance and capacity

The assignment does not prescribe measurable throughput or latency. The following are PROPOSED acceptance targets for a controlled academic demo and must be validated against the team's deployment environment.

ID	Requirement / target	Measurement
NFR-PERF-001	95th percentile latency for ordinary successful API reads/writes shall be ≤ 500 ms at 50 concurrent users, excluding external provider latency.	Load test report
NFR-PERF-002	A committed parcel status update shall become visible to connected tracking clients within 5 seconds under normal load.	Timestamped integration test
NFR-PERF-003	A committed outbox event shall normally be published to RabbitMQ within 5 seconds when broker is healthy and backlog is low.	Outbox created_at vs published_at
NFR-PERF-004	List endpoints shall paginate results and enforce a maximum page size.	API tests
NFR-PERF-005	AI requests shall have a configurable timeout and rate/cost limits; no fixed LLM latency guarantee is assumed.	Metrics and timeout tests

9.2 Availability and resilience

- Business mutations shall not depend on immediate notification-provider availability.
- Broker/provider failures shall be observable and recoverable through retry/reconciliation.
- API handlers shall use bounded timeouts and graceful shutdown.
- Worker replicas shall tolerate duplicate messages and process termination.
- Readiness checks shall prevent traffic routing to an instance that cannot currently serve requests.
- Stateful infrastructure recovery and backup/restore procedures shall be documented.

9.3 Maintainability

- Use Go formatting, static analysis, code review, and consistent package naming.
- Keep business logic separate from HTTP handlers, database access, and provider adapters.
- Publish OpenAPI and event schema contracts.
- Maintain migrations and example configuration in version control; do not commit secrets.
- Document local setup, architecture, environment variables, service ownership, and common failure recovery.

9.4 Usability and accessibility

- All primary workflows shall be responsive on mobile and desktop widths.
- Forms shall identify required fields, show actionable validation errors, and preserve safe user input after recoverable errors.
- Status shall be communicated using text as well as color.
- Keyboard navigation, visible focus, semantic labels, and reasonable contrast shall be supported.
- Public tracking shall not require login when accessed by a valid tracking number, but shall disclose only privacy-safe information.

9.5 Compatibility

The web client shall support current stable versions of major Chromium-based browsers, Firefox, and Safari where practical. SSE shall use native EventSource or an explicitly documented compatible client. Browser-specific behavior must be tested for the selected authentication strategy.

10. Event Processing and Failure Recovery

10.1 Status update transaction

- Authenticate actor and authorize the parcel/assignment.
- Load the parcel and verify the requested transition and current version.
- Within one database transaction: update current status/version; insert status history and attempt/proof metadata as applicable; insert outbox event.
- Commit transaction and return success to the caller.
- Outbox Relay later publishes the event to RabbitMQ and marks the outbox row published after broker confirmation.
- Event Worker consumes event and creates/processes notification and webhook delivery work idempotently.
- Tracking clients are notified through the configured SSE fan-out path; clients can re-fetch authoritative state.

10.2 RabbitMQ baseline

- Use durable exchanges and queues and persistent messages for business events.
- Use publisher confirms and manual consumer acknowledgments.
- Configure bounded retry/backoff and a DLQ/dead-letter route.
- Use stable event IDs and consumer idempotency records or unique constraints.
- Do not hold a PostgreSQL transaction open while waiting for an external webhook response.
- Configure prefetch/concurrency to prevent a slow provider from consuming unbounded memory or connections.

10.3 Reconciliation checks

Check	Detection	Recovery action
Stale outbox	Unpublished row older than configured threshold	Make eligible for safe retry, verify claim state, or alert; relay remains normal publisher
Stuck webhook delivery	Due retry time passed or delivery in processing beyond lease/timeout	Requeue safely if retry budget remains; otherwise mark investigation/DLQ
Missing history	Current parcel version/status has no corresponding history row	Flag for investigation; repair only with audited, deterministic procedure
Conflicting assignment	Multiple active assignments for one parcel	Flag and resolve through controlled transaction and audit
Consumer poison message	Repeated processing failure beyond retry budget	Dead-letter with event ID and failure metadata; alert

10.4 Delivery guarantees

The system provides durable at-least-once event processing under the configured PostgreSQL/RabbitMQ durability assumptions. Exactly-once side effects across an external email/SMS/webhook provider cannot be guaranteed generically. Use idempotency and provider idempotency support where available, and expose delivery state so duplicate/failed outcomes can be investigated.

11. Deployment and Operations

11.1 Local development

- Docker Compose shall start the dependencies needed for a developer to run the system locally, including PostgreSQL and RabbitMQ.
- Provide `.env.example` with placeholder values only.
- Provide migration and seed commands and a documented order for starting services.
- Use mock notification delivery and a local proof-storage substitute if external provider credentials are unavailable.
- Document Google OAuth redirect URLs and optional LLM configuration for local development.

11.2 Kubernetes deployment

- Deploy frontend, Core API, Tracking API, Outbox Relay, Event Worker, and AI Assistant using Deployments/Services as applicable.
- Provide ConfigMaps for non-sensitive settings and Secrets for credentials.
- Provide gateway/Ingress routes and TLS configuration for deployed environments.
- Configure startup, readiness, and liveness probes.
- Set CPU/memory requests and limits and replica counts appropriate to the environment.
- Use HPA for request-serving services where meaningful; use KEDA for RabbitMQ backlog-driven Event Worker scaling.
- Run reconciliation as a scheduled CronJob with a service account and least-privilege credentials.
- Provide Helm charts or equivalent repeatable manifests and environment-specific values.
- If PostgreSQL/RabbitMQ run in Kubernetes, provide persistent storage, backup, upgrade, and recovery notes; do not assume a Deployment alone makes stateful data safe.

11.3 Environment variables

Variable family	Examples / purpose	Secret?
APP_ENV / LOG_LEVEL	Environment and log verbosity	No
HTTP_ADDR / PUBLIC_BASE_URL	Listen address and external URL	No
DATABASE_URL or DB_*	PostgreSQL connection configuration	Yes if includes credentials
RABBITMQ_URL or RABBITMQ_*	Broker connection configuration	Yes if includes credentials
JWT_SIGNING_KEY / OIDC_CLIENT_SECRET	Token signing and Google client secret	Yes
GOOGLE_CLIENT_ID / OIDC_ISSUER	OIDC configuration	Client ID usually not secret; issuer no
WEBHOOK_SECRET_KEY_REF	Secret reference for webhook signing	Yes/reference
EMAIL_* / SMS_*	Notification provider settings	Often yes
LLM_API_KEY / LLM_MODEL	AI provider credentials/model selection	API key yes
METRICS_ADDR / OTEL_*	Metrics/tracing configuration	Usually no, unless exporter credentials
OUTBOX_STALE_SECONDS / RETRY_*	Recovery thresholds and retry policy	No
CORS_ALLOWED_ORIGINS / RATE_LIMIT_*	Security and gateway/application policies	No

11.4 Observability

- Expose liveness/readiness and metrics endpoints on an internal interface where possible.
- Record request count, latency histogram, error count, database pool saturation, outbox age/count, relay publish failures, RabbitMQ backlog, consumer failures, DLQ count, webhook delivery success/failure, notification outcomes, SSE connections, and AI request/cost/error metrics where available.
- Use correlation/request IDs across HTTP calls and event IDs across asynchronous processing.
- Never use tracking numbers, emails, full URLs containing secrets, or other high-cardinality personal identifiers as metric labels.
- Provide dashboards and an operational runbook for broker outage, DB outage, stuck outbox, webhook failures, and deployment rollback.

12. Testing and Verification

12.1 Test levels

Level	Tools / method	Required coverage
Unit	Go testing/testify; Python pytest; frontend Vitest	Pricing rules, transition rules, validation, permissions, idempotency, retry schedule
Integration	PostgreSQL/RabbitMQ test instances; Testcontainers where practical	Migrations, transactions, outbox relay, consumer idempotency, webhook retry
API/security	HTTP integration tests	Authentication, RBAC, tenant isolation, validation, error format, rate limiting
Frontend component	React Testing Library	Forms, error states, role-based navigation, tracking display
End-to-end	Playwright	Customer booking, merchant order, dispatch, courier update, tracking, receipt
Deployment	Docker Compose and Kubernetes acceptance tests	Configuration, probes, gateway routes, persistence, CronJob, scaling
Resilience	Fault injection/manual controlled tests	Broker unavailable, worker killed after publish, provider timeout, duplicate message

12.2 Mandatory acceptance tests

- AT-001: Customer registers/logs in and can create a valid parcel booking after seeing a fee estimate.
- AT-002: Invalid parcel inputs are rejected with field-level errors and no partial booking is stored.
- AT-003: Repeating a booking request with the same idempotency key creates only one parcel.
- AT-004: A customer cannot read another customer's private parcel history or receipt.
- AT-005: Public tracking displays status without exposing private address/contact information.
- AT-006: A valid courier can update an assigned parcel; another courier cannot update it.
- AT-007: An illegal status transition is rejected and creates no status history or outbox event.
- AT-008: A valid status change atomically creates status history and an outbox row.
- AT-009: The relay publishes the event after RabbitMQ becomes available and marks it published only after confirmation.
- AT-010: Duplicate RabbitMQ delivery does not duplicate the notification/webhook effect beyond the defined idempotency behavior.
- AT-011: Merchant order creation is rejected for unapproved/disabled merchants and is tenant-scoped.
- AT-012: Merchant receives a signed webhook for a configured event, with retry after a simulated transient failure.
- AT-013: Dispatcher assignment is concurrency-safe and workload counts update correctly.
- AT-014: Google login validates the OIDC flow and fails safely for invalid state/claims.
- AT-015: AI Assistant uses backend tools, respects authorization, and does not invent a status when tool data is unavailable.
- AT-016: Kubernetes readiness/liveness behavior and reconciliation CronJob are demonstrated if the bonus deployment is included.

12.3 Coverage and test reporting

The assignment requires measurable coverage and test results. BASELINE target: aim for at least 70% line coverage of core business packages, with explicit tests for every security/authorization requirement and the end-to-end event pipeline. Coverage percentage is not a substitute for meaningful assertions. Submit test commands, reports, coverage summary, known gaps, and environment details.

13. Design Decisions and Developer Contracts

13.1 Decisions fixed by this SRS baseline

- Core API is a Go modular monolith; modules do not communicate over HTTP internally.
- Tracking API is a separate Go deployment and uses read-only PostgreSQL access.
- PostgreSQL is authoritative for business state and outbox.
- RabbitMQ is the asynchronous broker; Outbox Relay and Event Worker are separate workloads.
- SSE is the initial browser live-update mechanism.
- AI Assistant uses Python/FastAPI and an LLM provider; MCP tools are initially packaged with the AI service.
- Reconciliation is a scheduled Go command, not a permanently running service.
- REST/JSON is the initial API contract; GraphQL and gRPC are not required.
- KEDA scales queue-driven workers; HPA scales request-serving workloads based on configured metrics.
- All consumers assume at-least-once delivery and implement idempotency.

13.2 Decisions the team must explicitly approve

Decision	Recommended baseline	Owner
FAILED status semantics	Separate delivery attempt outcome from parcel lifecycle; allow reattempt only through explicit dispatcher action	Domain/backend
Pricing formula and currency	LKR; rule-based zone/weight/dimension pricing; exact formula approved by product/team	Domain/backend
Authentication storage	Short-lived JWT + secure refresh/session approach; document browser token storage	Security/backend
Public tracking disclosure	Status and timestamps only; no private addresses/phones/recipient name	Product/security
Proof upload storage	Private object storage in production; local substitute for development	Backend/DevOps
SSE fan-out	Use shared fan-out such as Redis Pub/Sub if multiple replicas need cross-instance notification; DB remains source of truth	Backend/DevOps
Retry policy	Bounded exponential backoff with jitter; concrete attempt count and delays configurable	Backend/operations
Retention periods	Set retention for audit, delivery proof, webhook attempts, and personal data	Product/security
Performance targets	Approve Section 9.1 proposed targets after first benchmark	Team/QA
Gateway implementation	Kong Gateway/Ingress integration or another single documented implementation; avoid duplicate proxies without purpose	DevOps
Database/broker hosting	Compose locally; decide in-cluster vs managed for bonus/target environment	DevOps

13.3 Coding conventions

- Use clear module boundaries: handlers/controllers → application/use-case layer → domain logic → repository/adapters.
- Do not place SQL in HTTP handlers or business rules in frontend code.
- Do not call external email/webhook/LLM providers while holding business database transactions open.
- Use context cancellation and timeouts for all network/database operations.
- Return typed/domain errors mapped centrally to HTTP errors.
- Use migrations for schema changes; do not rely on auto-migration in production.
- Version event schemas and maintain backward compatibility for active consumers.
- Use interfaces at meaningful external boundaries, not for every trivial struct.
- Every requirement ID should map to one or more test cases in the traceability matrix.

14. Risks and Mitigations

Risk	Impact	Mitigation
Duplicate messages	Duplicate notifications or webhooks	Stable event IDs, idempotency records/unique constraints, provider idempotency when supported
Outbox backlog	Delayed downstream events	Metrics on oldest unpublished event, bounded batches, relay replicas, reconciliation and alerts
Shared PostgreSQL overload	Tracking latency affects booking	Indexes, query/pool limits, separate Tracking API, load testing; consider read replica only if measured
Public tracking enumeration	Privacy leakage	High-entropy tracking numbers, rate limiting, privacy-safe fields, anomaly monitoring
Cross-tenant access bug	Merchant data disclosure	Tenant scoping in repository/service layer, authorization tests, DB constraints where useful
Webhook SSRF	Internal network access from worker	Strict URL validation, IP/redirect controls, egress restrictions, DNS rebinding considerations
External provider outage	Delayed notifications/webhooks	Asynchronous processing, retries, DLQ, operational visibility
LLM hallucination or prompt injection	Incorrect information or unauthorized action	Read-only tools, authoritative backend data, argument validation, authorization preservation, no direct DB access
Kubernetes complexity	Delivery delays and operational burden	Build with Compose first; use Helm/manifests, probes, resource limits, and incremental deployment
Unclear failed-delivery semantics	Invalid status transitions and retry bugs	Approve the explicit attempt/state model before implementing courier workflows

15. Project Priorities and Acceptance

15.1 Priority definitions

- P1 — required for core assignment demonstration and safe baseline.
- P2 — bonus or secondary feature; implement after P1 workflows are reliable.
- P3 — future enhancement, not required by this baseline.

15.2 P1 completion criteria

- Customer signup/login including Google login; booking with price estimate; tracking and receipt.
- Merchant approval, secured order creation API, dashboard, and webhook delivery with retry.

- Courier assigned-delivery view, status updates, delivery proof/note, failed reason.
- Dispatcher assignment/workload and admin user/merchant/zone/pricing management plus basic reports.
- RBAC and object/tenant authorization tests.
- PostgreSQL transactional outbox, RabbitMQ, notification consumer, webhook worker, and failure handling.
- Responsive frontend, versioned documented APIs, Dockerfiles, and Docker Compose.
- Unit/integration tests including at least one end-to-end event flow, coverage report, architecture/design documentation, README, and contribution sheet.

15.3 Bonus completion criteria

- Kubernetes deployment of frontend, backend workloads, database and broker as required by the assignment; ConfigMaps, Secrets, Services, and Ingress/Gateway configuration demonstrated; Helm is recommended.
- AI Assistant answers tracking and fee questions by calling backend APIs; MCP tool exposure is recommended.
- Demonstrate HPA/KEDA configuration and reconciliation CronJob where included in the deployment scope.

15.4 Defect priority

Priority	Definition	Example
P0	Critical security/data loss or system cannot demonstrate core flow	Cross-tenant data exposure; all bookings fail
P1	Core feature incorrect or insecure; no viable workaround	Courier can update unassigned parcel
P2	Secondary defect with workaround	Report filter edge case
P3	Minor usability/documentation improvement	Cosmetic alignment issue

16. Requirements Traceability Matrix

Assignment requirement	SRS coverage
Customer signup/login + Google	FR-IAM-001 to FR-IAM-009; Section 6.2
Create delivery and estimate fee	FR-PAR-001 to FR-PAR-006; Section 5.2
Track real time and history	FR-TRK-001 to FR-TRK-007; FR-LIFE-001 to FR-LIFE-007
Receipt download	FR-PAR-008
Merchant approval/API/webhooks/dashboard	FR-MER-001 to FR-MER-011
Courier assigned deliveries/status/proof/failure	FR-COU-001 to FR-COU-007
Dispatcher assignment/workload	FR-ADM-001 to FR-ADM-004
Admin users/merchants/zones/pricing/reports	FR-ADM-005 to FR-ADM-009
RBAC/JWT/OAuth	FR-IAM-003 to FR-IAM-009; SEC-001 to SEC-017
Broker/status events/notifications/webhook retry	FR-EVT-001 to FR-EVT-010; Section 10
Containerization/Compose	Sections 11.1 and 11.2
Unit/integration/security/event testing	Section 12
Design diagrams and principles	Sections 3, 10, 13; companion design documentation required
Kubernetes bonus	Sections 11.2 and 15.3
AI assistant/MCP bonus	FR-AI-001 to FR-AI-008

17. Deliverables and Open Items

17.1 Required repository deliverables

- Source code and README with prerequisites, local setup, migrations, environment variables, test commands, and troubleshooting.
- Architecture diagram, use-case diagrams, ER diagram, and sequence/activity diagrams for booking, merchant order, courier status/event flow, and Google OAuth.
- OpenAPI specifications and event/webhook schema documentation.
- Database migrations and seed/demo data scripts.
- Dockerfiles and Docker Compose configuration; Kubernetes manifests/Helm chart if bonus is attempted.
- Automated test suites and test/coverage reports.
- Security notes, deployment/runbook documentation, and a contribution sheet for all group members.

17.2 Open decisions before implementation freeze

- Approve the failed-delivery/reattempt state machine.
- Approve the fee formula, currency representation, service levels, and zone model.
- Select the exact JWT/session/refresh design and token lifetimes.
- Select email/SMS provider or mock implementation and define retry schedule.
- Set data-retention periods and proof-file policy.
- Choose the gateway deployment shape and cross-replica SSE fan-out.
- Approve proposed performance targets after initial measurement.
- Decide whether PostgreSQL/RabbitMQ are deployed inside Kubernetes or supplied by managed services in the target environment.

17.3 Change control

Changes to requirements shall be recorded with requirement ID, rationale, impact on API/data/event contracts, affected tests, owner, and approval. Breaking API/event schema changes require versioning or a documented coordinated migration. This SRS is a baseline; changes should be reviewed by the team before implementation diverges.